



Abstract

Transformer 아키텍처는 이미 NLP 분야에서는 사실상 표준이다. 근데 비전 task에서는 어떨까? attention 매커니즘이 CNN이랑 결합되는 형식으로, 즉 CNN의 전체적인 구조는 유지되는 형태로 사용되었다.

이 논문은 CNN 의존이 반드시 필요하지 않음을 보여준다.

대규모 데이터로 pre training한 후 이미지 인식 벤치마크에 전이 학습을 하면 좋은 성과를 보여준다!

Introduction

컴퓨터 비전 분야에서는 여전히 CNN 아키텍처가 지배적이다. 물론 NLP에서의 Transformer의 영향에 의해 CNN에 Self-attention을 결합해보는 시도는 있었지만 이는 특수한 어텐션 패턴을 사용하기 때문에 확장성 면에서 좋지는 않았다.

그래서 그런지 아직 대규모 이미지 인식 쪽에서는 ResNET 계열이 SOTA를 기록하고 있다.

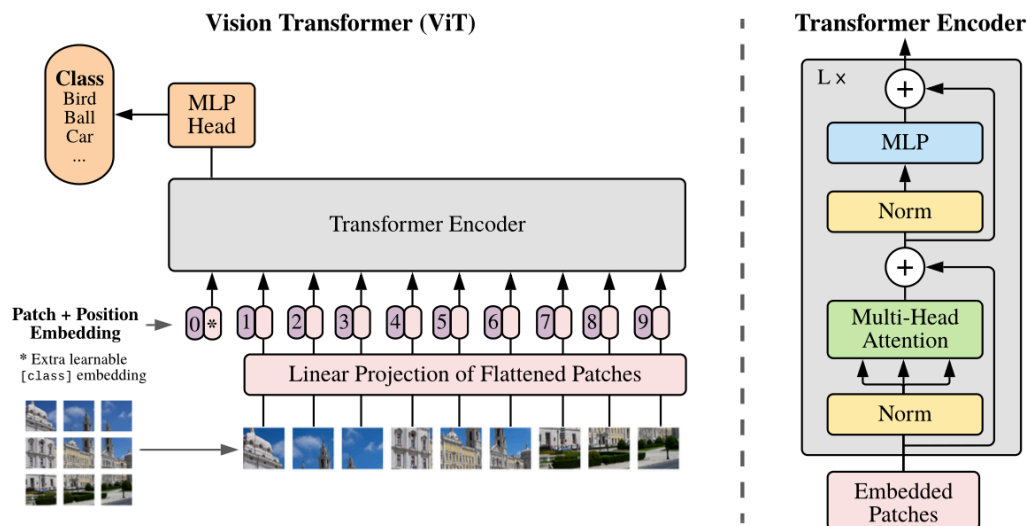
이 논문에서는 표준 Transformer를 직접 이미지에 적용하는 실험을 진행한다.

- 이미지를 패치로 나누고 → 각 패치를 선형 임베딩하여 얻은 시퀀스를 입력으로 사용한다.
- 이러한 패치는 마치 NLP과제 에서 단어 토큰에 비유할 수 있다.
- 모델은 이미지 분류 작업에 대해서 지도학습을 사용한다.

ImageNet과 같은 중간 규모의 데이터에서는 Transformer 보다는 ResNet이 성능이 좋다
이러한 이유는 **translation equivariance, locality**같은 **inductive bias**를 갖고 있지 않기 때문에 **충분히 많은 데이터로 학습하지 않으면 일반화 성능이 떨어지기 때문이다.**

하지만 대규모 데이터를 학습할 때는 상황이 달라진다. 를 대규모 학습이 inductive bias를 성능 면에서 더 좋은 학습을 이끌어 낸다는 점을 발견했다.

Method



전체 이미지 $x \in \mathbb{R}^{H \times W \times C}$ 를 패치로 분할하여 각 패치 $x_p \in \mathbb{R}^{N \times (P^2 \times C)}$

총 패치의 개수 N 은 HW/P^2 과 같다. 이것이 곧 transformer의 입력 시퀀스 길이가 될 것.

latent vector size D 로 transformer에 넣어야 학습 가능하므로 linear projection을 하여 D 차원으로 만들어준다.

BERT와 유사하게 [class] 토큰을 사용한다.

[cls] 토큰을 패치 시퀀스의 헤드에 붙인다. 이 토큰은 학습 가능한 패치 임베딩임을 유념하자.

pre-training, fine-tuning 시에는 이 cls 토큰의 output에 연결되어 이미지 표현 y 로 사용된다.

다만 pre-training시에는 은닉층을 가진 MLP 레이어를 붙여 분류하고

fine-tuning시에는 그냥 linear layer로 구현하여 분류한다.

위치 정보를 유지하기 위해 패치 임베딩에 position 임베딩을 추가한다. 이 논문에서는 1D position embedding 을 사용했는데 2D-aware position embedding을 사용해도 그다지 큰 성능 향상을 관측하기 못했기 때문이다.

2D aware은 패치의 행,열 위치까지 고려한거고, 1D는 단순하게 패치를 일렬로 나열한다음 위치 임베딩을 한 것

transformer encoder는 Multi-Head Self Attention과 MLP를 계속해서 쌓는 구조로 되어있다. 두 블록 앞에는 Layer Normalization이 적용되고 두 블록 뒤에는 잔차연결이 적용된다.

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D} \quad (1)$$

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \ell = 1 \dots L \quad (2)$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \quad \ell = 1 \dots L \quad (3)$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0) \quad (4)$$

다시 한 번 정리하자면

(1) : 각 이미지 패치를 D차원으로 투영하고 class토큰도 넣고 위치 임베딩도 넣어주고해서 시퀀스를 만든다.

(2) : Layer Normalization에 넣고 MSA연산도 하고 잔차연결

(3) : Layer Normalization에 넣고 MLP연산도 하고 잔차연결

(4) : class 토큰 벡터에 LN 적용 이것이 최종출력 y이다.

Inductive bias

지역성(locality), 2차원 이웃 구조(2D neighborhood structure), 그리고 평행이동 등가성(translation equivariance)

과 같은 inductive bias가 CNN에 비해 훨씬 적다.

MLP 계층만이 지역성, 평행이동 등가성을 가지며, self attention 계층은 전역적이다.

2차원 이웃 구조에 대해서.

이미지를 패치로 나눌때, 서로 다른 해상도의 이미지를 처리할 때 위치 임베딩은 조정하는 fine-tuning 과정에서만 제한적으로 사용

초기화 시의 위치 임베딩은 패치의 2차원 위치에 대한 정보는 없음

패치 간 모든 공간적 관계는 학습 과정에서 처음부터 배워야 한다.

Hybrid Architecture

입력 시퀀스를 이미지 패치를 그대로 사용하는게 아닌 CNN의 feature map으로부터 만들어 낼 수도 있다. 투영 행렬 E를 CNN의 Feature map에 적용한다고 생각하면 된다. 그 외에 class 토큰이나 뭐 위치임베딩 방식은 동일하다.

3.2 FINE-TUNING AND HIGHER RESOLUTION

사전 학습을 진행한 후, 더 작은 다운스트림 task에 대해 파인튜닝을 진행하는데 이때 사전 학습된 헤드를 제거하고 $D \times K$ 크기의 feedforward layer를 새로 붙인다. 단, K는 다운스트림의 클래스 개수 이고 이 계층은 0으로 초기화 된다.

파인튜닝시에 근데 일단 더 높은 해상도에서 학습하는 것이 유리할 때가 많다

근데 큰 해상도의 이미지를 입력할 때 패치크기는 유지되므로 시퀀스 길이가 길어진다. 물론 시퀀스 길이 자체는 임의값이라 상관은 없다만 사전학습된 위치임베딩이 의미를 잃는 경우가 생긴다. 그래서 사전 학습된 위치 임베딩을 원본 이미지 내 위치에 따라 2D 보간 하여 조정한다.

Experiments

ResNet, ViT, Hybrid 세 모델의 학습 능력을 평가했다.

- 각 모델이 필요로 하는 데이터량은 얼마일까? → 다양한 크기의 데이터 셋, 벤치마크 과제로 사전 학습 진행

계산 비용을 고려했을 때 ViT는 매우 긍정적인 성능을 보였다. 대부분의 Recognition 벤치마크에서 SOTA 결과를 더 낮은 사전학습 비용으로 달성할 수 있었다.

self-supervision 을 이용한 작은 실험을 수행했고 가능성을 보았다.

4-1 SETUP

DataSets

pre-training 데이터 셋

- ImageNet(1K classes, 1.3M images)
- ImageNet-21k(21K classes, 14M images)
- JFT(18K classes, 330M images)

De-duplication

사전학습과 다운스트림 테스트셋 겹치는 사진은 제거

다운스트림 벤치마크 데이터 셋

- imageNet
- CIFAR-10/ CIFAR-100
- Oxford-IIIT Pets
- Oxford Flowers - 102

또한 19개의 작업으로 구성된 VTAB(Vision Transfer Assessment Benchmark)를 수행
각 task당 1000개의 학습 예제를 사용해 저데이터 전이 성능을 평가한다.

- **Natural** – Pets, CIFAR 등 위에서 언급한 일반적인 자연 이미지 과제들
- **Specialized** – 의료 영상, 위성 영상과 같은 특수 도메인 과제들
- **Structured** – 위치 추정(localization) 등 기하학적 이해를 요구하는 과제들

Model Variants

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

BERT의 사용구성에 기반하여 BASE LARGE 모델 구성하였고 이보다 더 큰 HUGE 모델을 추가하였다.

또한 /XX 로 되어있는 뒤의 숫자는 XX x XX 의 입력 패치 크기를 사용한다는 뜻이다.

시퀀스 길이는 패치크기의 제곱에 반비례하므로 패치 크기가 작을 수록 모델은 계산 비용이 더 많이 든다는 점을 생각하자

비교를 위한 CNN으로는 ResNet을 사용하되, Batch Normalization 레이어를 Group Normalization으로 교체하고 standardized convolutions을 사용했다. 이를 ResNet(BiT)라 칭한다.

Hybrid 모델의 경우 ResNet 중간 feature map을 ViT에 입력하며, 이때 패치 크기는 10이다.

1. 일반 ResNet50의 stage4 (feature map)을 사용
2. stage 4를 제거하고 같은 레이어 수를 stage 3에 추가한 뒤 이 확장된 stage 3 출력을 사용 → 시퀀스의 길이가 4배 더 길어지므로 ViT 모델이 더 계산 비용상승

Training & Fine-tuning

모든 모델을 Adam 옵티마이저를 사용 → ResNet 계열로 확인해봤더니 Adam 이 SGD 보다 더 나은 성적을 보인 것을 확인했다.

pre-training

Models	Dataset	Epochs	Base LR	LR decay	Weight decay	Dropout
ViT-B/{16,32}	JFT-300M	7	$8 \cdot 10^{-4}$	linear	0.1	0.0
ViT-L/32	JFT-300M	7	$6 \cdot 10^{-4}$	linear	0.1	0.0
ViT-L/16	JFT-300M	7/14	$4 \cdot 10^{-4}$	linear	0.1	0.0
ViT-H/14	JFT-300M	14	$3 \cdot 10^{-4}$	linear	0.1	0.0
R50x{1,2}	JFT-300M	7	10^{-3}	linear	0.1	0.0
R101x1	JFT-300M	7	$8 \cdot 10^{-4}$	linear	0.1	0.0
R152x{1,2}	JFT-300M	7	$6 \cdot 10^{-4}$	linear	0.1	0.0
R50+ViT-B/{16,32}	JFT-300M	7	$8 \cdot 10^{-4}$	linear	0.1	0.0
R50+ViT-L/32	JFT-300M	7	$2 \cdot 10^{-4}$	linear	0.1	0.0
R50+ViT-L/16	JFT-300M	7/14	$4 \cdot 10^{-4}$	linear	0.1	0.0
ViT-B/{16,32}	ImageNet-21k	90	10^{-3}	linear	0.03	0.1
ViT-L/{16,32}	ImageNet-21k	30/90	10^{-3}	linear	0.03	0.1
ViT-*	ImageNet	300	$3 \cdot 10^{-3}$	cosine	0.3	0.1

linear learning rate warmup, linear decay 를 사용했다.

우선 ImageNet 데이터 셋 자체가 크기가 큰 데이터 셋이 아니다. 따라서 ViT 방식을 사용할 경우 아무래도 과적합의 위험이 있다 따라서 강한 정규화를 적용한다.

Fine-tuning:

Dataset	Steps	Base LR
ImageNet	20 000	{0.003, 0.01, 0.03, 0.06}
CIFAR100	10 000	{0.001, 0.003, 0.01, 0.03}
CIFAR10	10 000	{0.001, 0.003, 0.01, 0.03}
Oxford-IIIT Pets	500	{0.001, 0.003, 0.01, 0.03}
Oxford Flowers-102	500	{0.001, 0.003, 0.01, 0.03}
VTAB (19 tasks)	2 500	0.01

파인튜닝 하이퍼파라미터. 모든 모델은 코사인 학습률 감소(cosine decay), 배치 크기 512, weight decay 없음, 그리고 전역 노름(global norm) 1에서의 gradient clipping으로 파인튜닝된다. 별도로 언급되지 않는 한, 파인튜닝 해상도는 384이다.

- 여기서 코사인 학습률 감소(cosine decay)는 학습이 진행될 수록 러닝레이트를 감소시키는 것.
- 사전학습 해상도 : 224 → 파인튜닝 해상도 : 384 (더 높게 쓰는 경우 있음)

다만 파인튜닝 시에는 0.9의 모멘텀을 가지는 SGD를 사용 → 왜 여기서 갑자기 Adam이 아니라 SGD를 사용하지?

▼ 관련 논문

관련 연구

1. Wilson et al. (2017), *"The marginal value of adaptive gradient methods in deep learning"*
 - Adam이 빠르게 수렴하지만, 일반화 성능은 SGD보다 떨어지는 경우가 많음을 실험적으로 보여줌.
2. Keskar & Socher (2017), *"Improving generalization performance by switching from Adam to SGD"*
 - Adam으로 pre-train 한 후, fine-tuning은 SGD로 전환하는 것이 더 좋은 일반화 성능을 준다는 연구.

전이 시에는 전체 헤드 레이어(두개의 레이어)를 제거하고 데이터 셋의 클래스 수를 출력하는 단일 선형층으로 교체한다. 그리고 이 층은 0으로 초기화한다. 이 방식이 마지막 층만 초기화하는 것보다 견고한 것을 발견했다.

VTAB은 BiT 논문 방식 그대로 사용(위에 테이블 참고) ViT의 경우 테스트마다 해상도를 변경하지 않고 384로 고정하는게 더 효율적임을 발견했다.

SGD with momentum, 배치 크기 512를 사용

Higher resolution fine-tuning 적용:

- ViT-L/16 → 512×512
- ViT-H/14 → 518×518

Polyak & Juditsky (1992) averaging을 계수 0.9999와 함께 적용

Metrics.

다운스트림 데이터셋에서 결과를 **few-shot** 또는 **fine-tuning accuracy**로 측정한다..

- Fine-tuning accuracy는 각 모델을 해당 데이터셋에서 파인튜닝한 뒤의 성능을 측정한다.

- Few-shot accuracy는 정규화된 최소제곱 회귀(regularized least-squares regression) 문제를 풀어 얻는다. 이는 학습 이미지 하위 집합의 고정된(frozen) 표현을 $\{-1, 1\}^K$ 타깃 벡터로 매핑하는 방식이다. 이 수식은 closed form으로 정확히 풀 수 있다. → 정답을 1로 보고 오답을 -1로 본다는 뜻

주로 fine-tuning 성능에 집중하지만, fine-tuning이 계산적으로 너무 비싼 경우 빠른 평가를 위해 선형 few-shot accuracy를 활용하기도 한다.

4.2 COMPARISON TO STATE OF THE ART

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

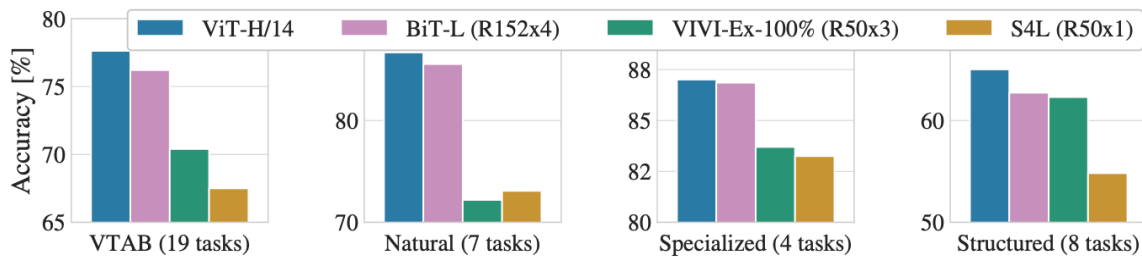
비교 대상

- ViT-H/14 - JFT 데이터셋
- ViT-L/16 - JFT 데이터셋
- ViT-L/16 - imageNet21k 데이터셋(위보다 적음)
- BiT-L
- Noisy Student - 반지도 학습으로 대규모 EfficientNet을 학습시킨 것.

다운스트림 태스크 뿐만 아니라 TPUv3 core days 즉 사전학습에 소요된 자원값도 고려한다.

- 작은 ViT-L/16 모델은 동일한 데이터셋으로 사전 학습된 BiT-L을 모든 작업에서 능가하면서, 훈련에 필요한 계산 자원도 훨씬 적게 소모했다.

- 더 큰 모델인 ViT-H/14는 성능을 한층 더 끌어올렸으며, 특히 ImageNet, CIFAR-100, VTAB suite에서 성능이 두드러졌다.
- ViT-H/14는 기존 state-of-the-art보다 사전 학습에 필요한 연산량이 훨씬 적었다.



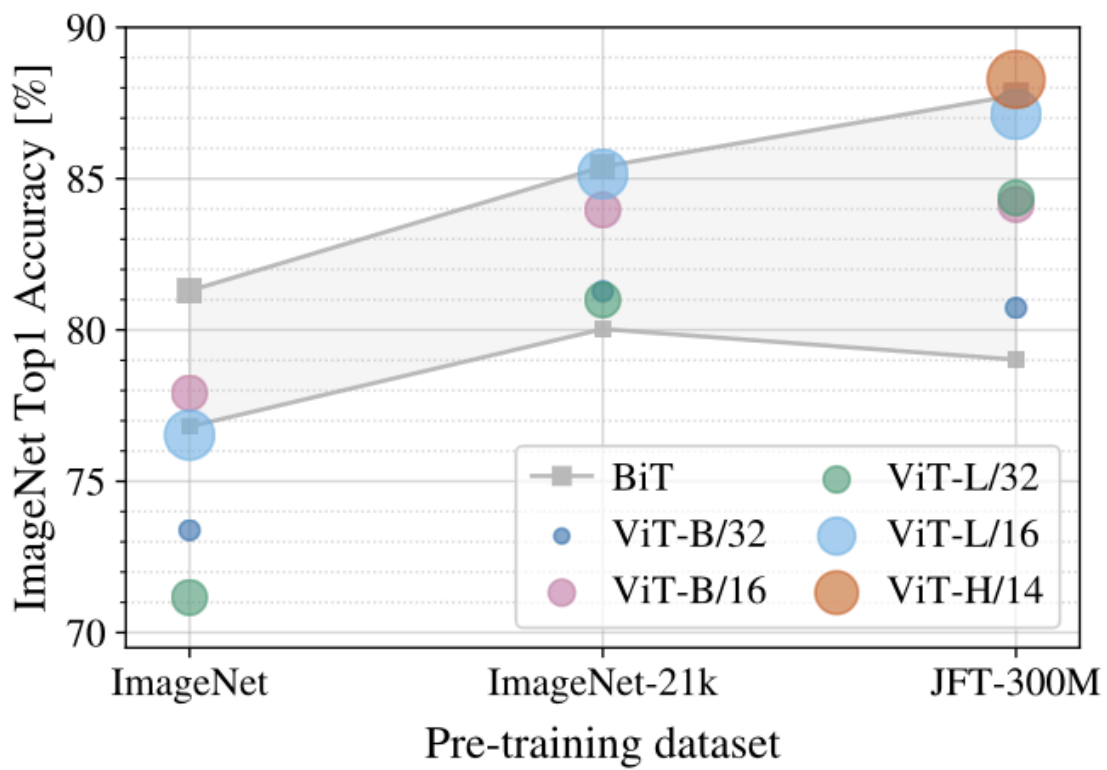
VTAB 으로 한번 더 들어가자면

ViT-H/14는 Natural 과제와 Structured 과제에서 BiT-R152×4 및 다른 방법들을 능가하였다. Specialized 과제에서는 상위 두 모델의 성능이 비슷하게 나타났다.

4.3 PRE-TRAINING DATA REQUIREMENTS

방금 위에서 보았듯 대규모 JFT-300M 데이터셋에서 사전 학습할 때 좋은 성능을 낸다. 하지만 ResNet보다 비전에 대한 inductive bias가 적은 ViT의 경우 과연 데이터셋의 크기가 얼마나 중요할까?

이에 대한 두 가지 실험을 진행한다.



기존에 언급했던 세가지 사전학습 데이터로 실험 진행

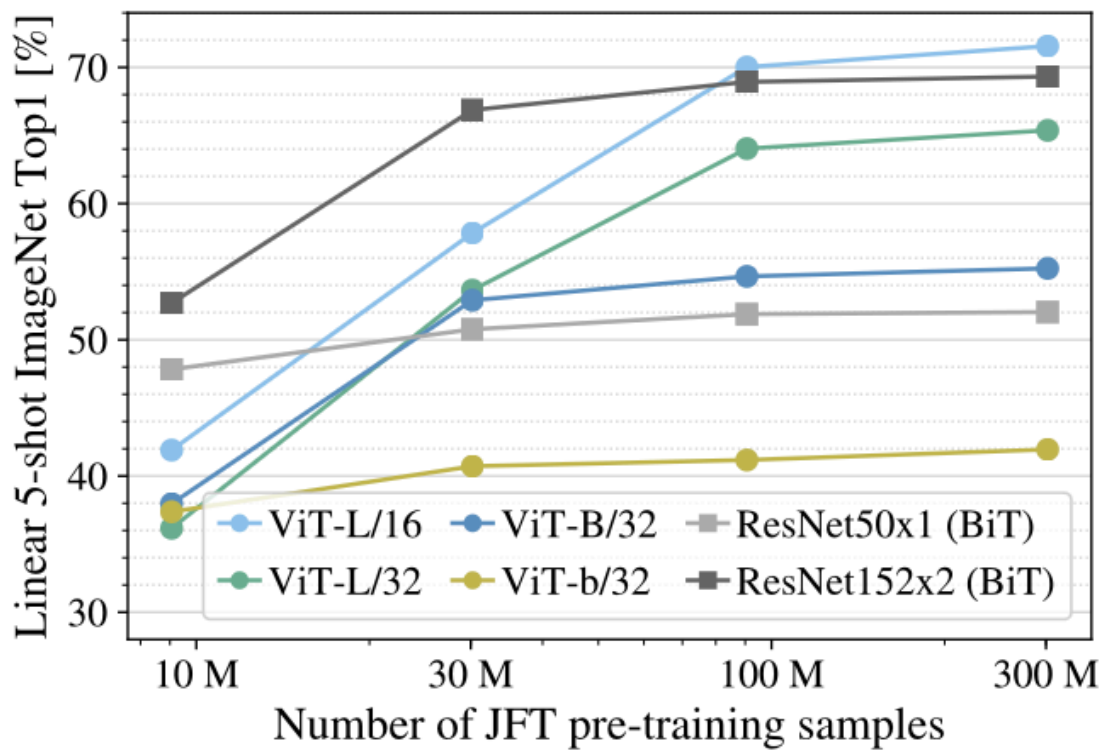
오히려 적은 데이터셋에서는 모델의 크기가 클 수록 더 나쁜결과

JFT 300정도는 와야 확실한 결과를 보여줌

회색 BiT영역은 모델 크기에 따른 성능영역

이 역시 데이터 셋의 크기가 클 수록 더 높은 성과를 보여주는 ViT를 볼 수 있다.

아무래도 inductive bias가 약하기 때문이 아닐까?

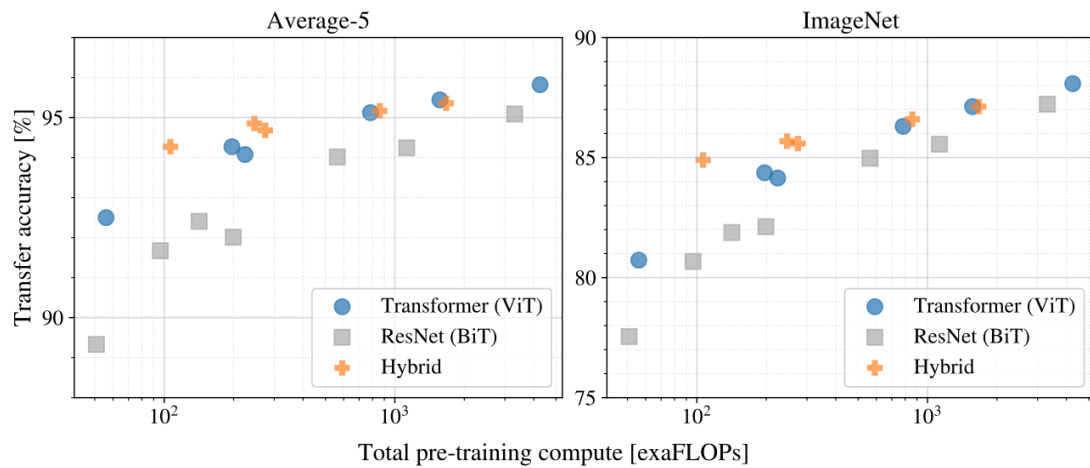


9, 30, 90 300 에서 추가적인 정규화 없이 동일 파라미터에서 실험을 진행했다. 이를 통해 정규화의 효과가 아니라 모델 자체의 고유한 특성을 평가할 수 있다.

작은면 과적합 가능성이 높기 때문에 정규화가 필요하지만 여기서는 그렇게 진행하지 않은 것이다.

결과를 보면 ViT가 작은 데이터 셋에서 ResNet보다 더 과적합 된 것을 볼 수 있다.

Scaling study



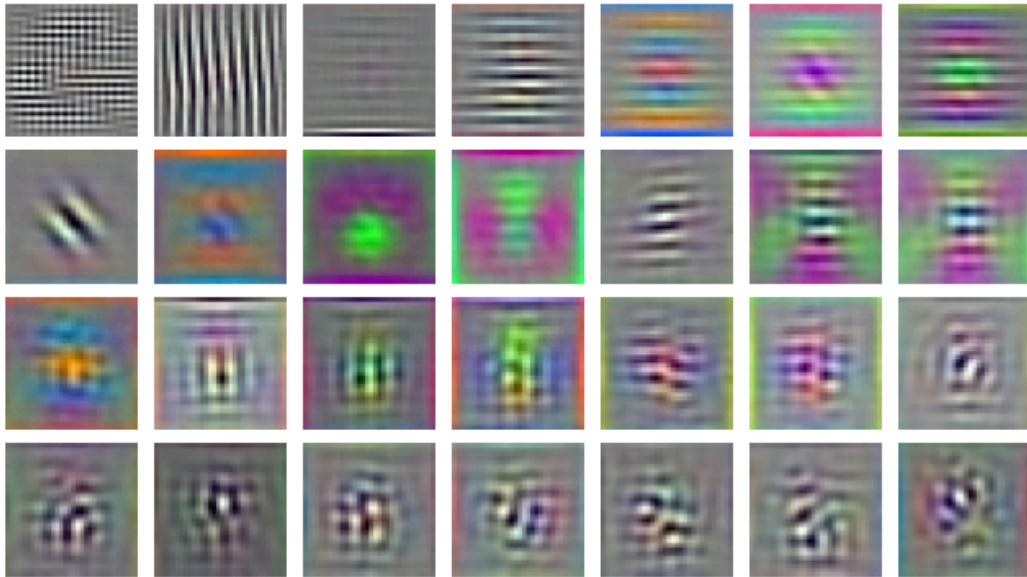
같은 데이터 셋으로 연산량과 비교 실험이다.

기본적으로 BiT보다 ViT계열이 더 연산량, 성능 측면에서 좋은 모습을 보여주고 있고

ViT에 ResNet을 결합시킨 모델은 사전학습 데이터량이 적을 때는 더 높은 성능을 보여주지만 점점 대규모 모델에서 부터는 그 격차가 줄어드는 점을 볼 수 있다.

INSPECTING VISION TRANSFORMER

RGB embedding filters
(first 28 principal components)



자 일단 우리는 패치 하나를 flatten 시키고 선형 투영 시켜 하나의 임베딩 벡터 형태로 만들고 이를 이용하는 건 이미 알고 있다.

이때 바로 이 E 라는 Projection 행렬에 대체 어떤 정보가 담겨서 이런 행위가 이루어지는가를 알고싶다.

그래서 PCA를 이용하여 E 의 기저 벡터, 기저 함수를 알아본다.

앞서 논문에서 사용한 모델은 ViT-L/32

즉 패치는 $32 \times 32 \times 3$ 의 정보를 가질 것 이며 hidden size D 는 1024이다.

이를 PCA해서 1024개의 주성분 축 중에서 상위 28개를 선택하고 다시 $32 \times 32 \times 3$ 패치 이미지 형태로 변경한 것이 위의 그림이다.

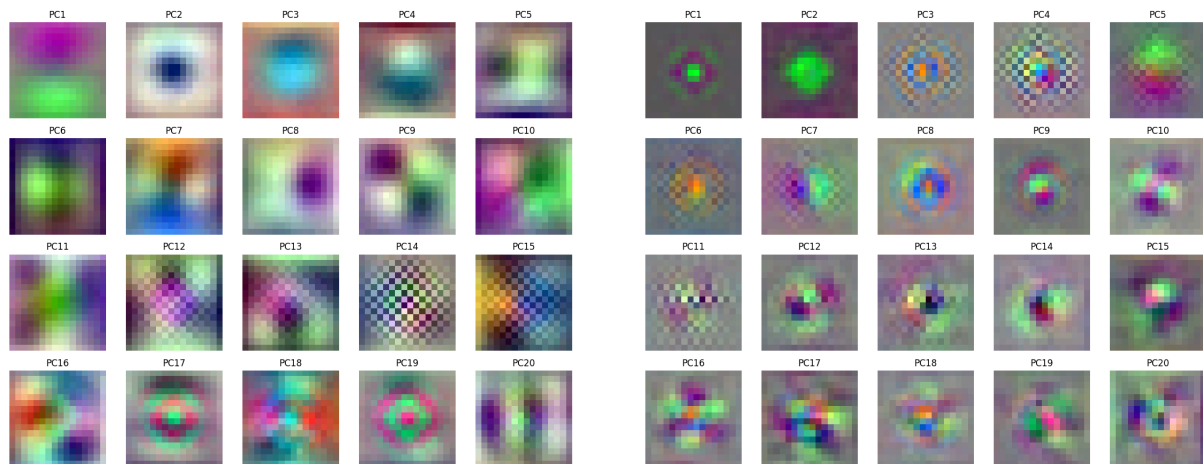
이것이 왜 의미있는가??

일단 ViT와 같은 트랜스포머 구조가 기존의 CNN 기반 아키텍처 처럼 학습 후에 시각적 기본 요소들의 필터가 존재하며 이를 사용한다는 것이다.

하지만 ViT는 구조적으로 inductive bias가 없다. 그런데 어떻게 저 시각적 기본 요소를 얻어낼 수 있는가?

이때 필요한 것이 방대한 양의 데이터 셋이다.

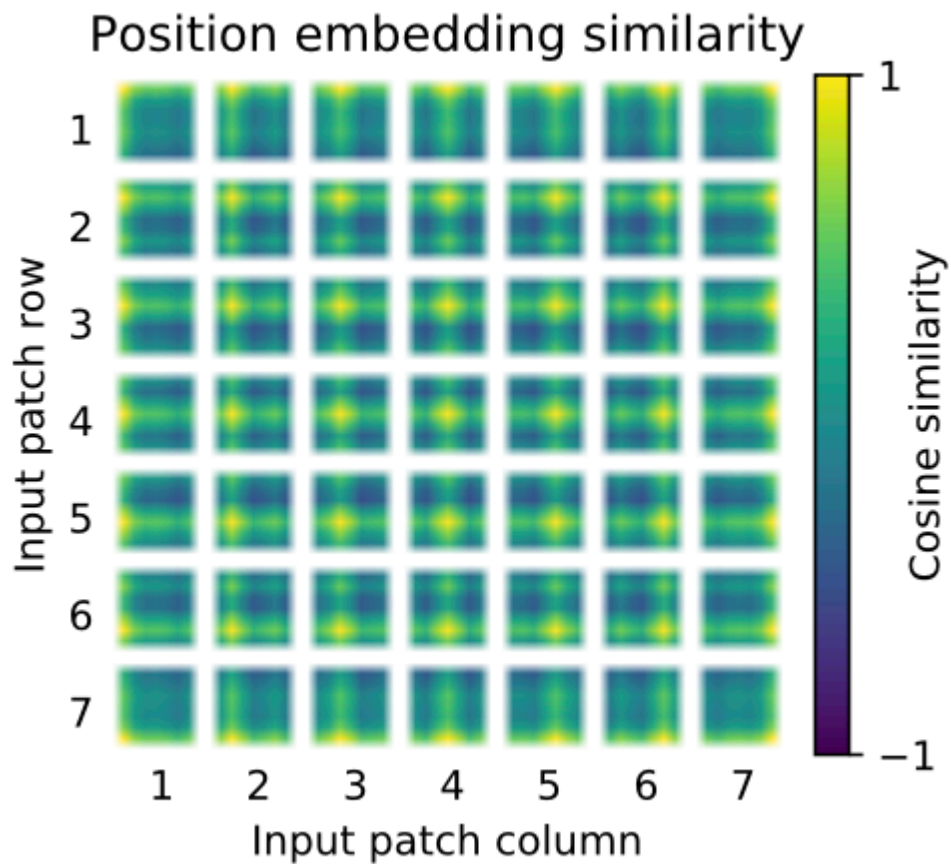
이러한 방대한 양의 데이터 셋을 통해서 어찌보면 귀납적으로 시각적 기본 요소를 스스로 깨우친다고 보면된다.



왼쪽은 L/16

오른쪽은 S/16

더 큰 모델에서 더 구체적인 시각적 특징을 가지고있다.



이전에 구조에서 2D 임베딩을 주지 않았다. 이미지인데 사실 이미지에서 어떤 사물의 위치나 뭐 그런것들은 굉장히 중요한 정보지 않을까?? 사람의 코가 얼굴 가운데에 있어야지 갑자기 이상한데 달리면 안되니까

그럼 어찌보면 그냥 패치의 위치 임베딩을 2D로 줘서 확실하게 위치를 알려주는 것이 이득이지 않을까? 라는 생각을 하게 된다.

이런 생각이 들지만 논문에서는 1D 임베딩을 사용하였다. 저자 말에 따르면 1D 위치 임베딩을 하던 2D 위치 임베딩을 하던 그다지 큰 성능 차이는 존재하지 않았다고 한다.

일단 임베딩 방식 4가지와, 임베딩 적용 방식 3가지 총 12가지를 이용해 실험한다.

임베딩 방식 4가지

1. 위치 임베딩을 적용하지 않는다.

2. 1차원으로 펼쳐서 위치 임베딩 적용한다. (좌상 → 우하)
3. 위치 임베딩을 두 축으로 나눠 적용한다 2D
4. 패치끼리의 상대적인 위치를 반영하면서 연산 시킨다 (상대적 임베딩)

임베딩 적용 방식 3가지

1. Transformer encoder 입력하기 전에 위치 임베딩을 더하기
2. 각 레이어의 시작 부분마다 각각 새로운 위치 임베딩을 학습하고 매번 더하기
3. 각 레이어의 시작 부분마다 위치 임베딩을 더하지만 이 임베딩들은 서로 공유된다.

Pos. Emb.	Default/Stem	Every Layer	Every Layer-Shared
No Pos. Emb.	0.61382	N/A	N/A
1-D Pos. Emb.	0.64206	0.63964	0.64292
2-D Pos. Emb.	0.64001	0.64046	0.64022
Rel. Pos. Emb.	0.64032	N/A	N/A

위와 같은 결과를 보여준다. 일단 위치임베딩을 적용하지 않았을 경우에는 확실히 나쁜 성능을 보였다.

다만 나머지 위치임베딩은 그다지 큰 성능 차이가 보이지 않는다.

차라리 1D가 적용하기 쉬우니 이를 활용하면 되는 것.

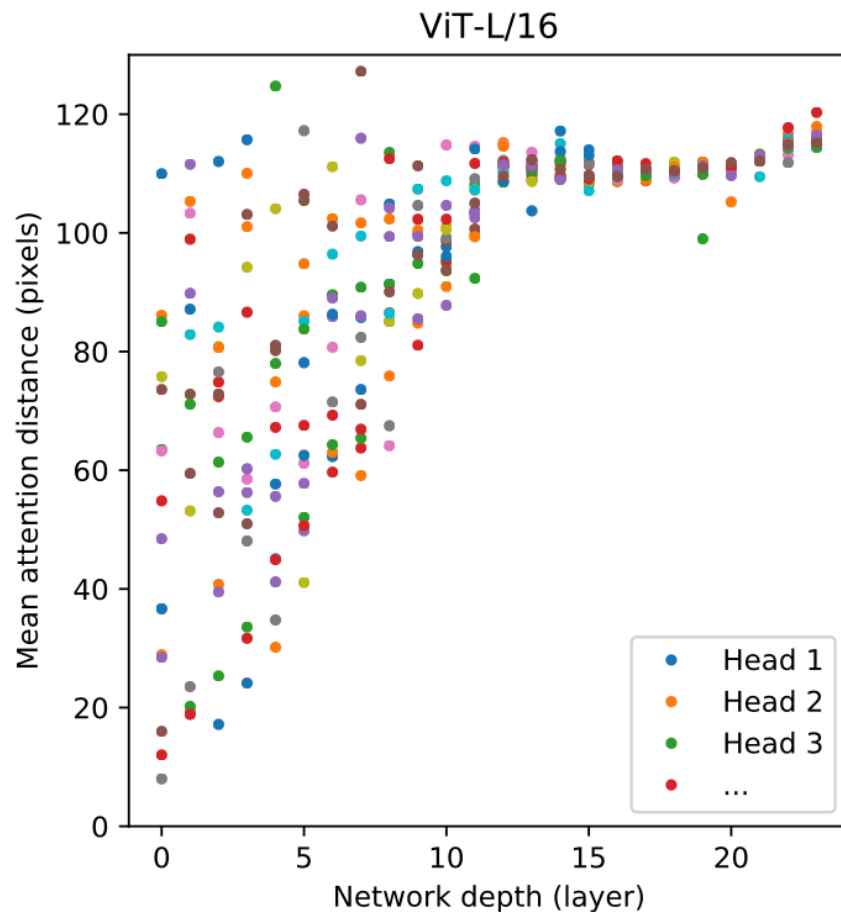
그래서 왜 이렇게 차이가 나지 않을까?

생각해보자면 이미지의 패치가 그다지 많지 않기 때문이 아닐까 라고 논문은 말한다.

우리가 생각하기에 엄청나게 긴 글에 임베딩을 적용하려면 사실 그 토큰은 아주 많을 것이다. 하지만 여기서 취한 방식에 따르면 $224 * 224$ 의 이미지를 사용하더라도 $16*16$ 의 패치를 사용하는 순간 $14*14 = 196$ 개의 임베딩 토큰, 벡터를 사용한다고 보면된다. 그 어떤 임베딩이라도 어렵게 처리하지는 않는다는 뜻

ViT의 self attention은 CNN 구조와 다르게 낮은 층의 레이어에서도 전역적인 정보를 활용할 수 있게 한다. 우선 전역적으로 접근한다는 사실을 알아내보자

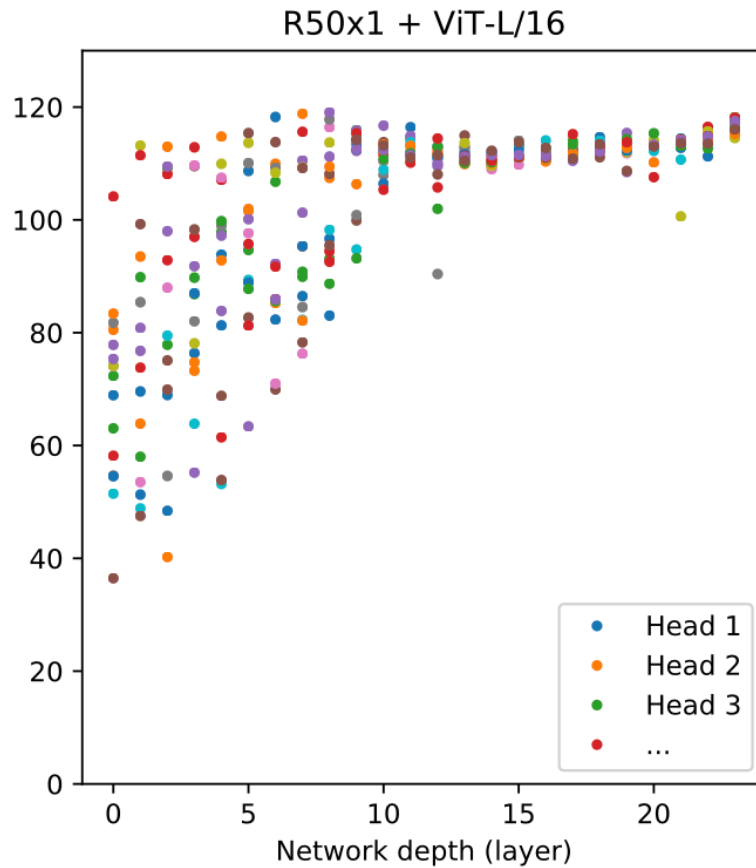
attention weights를 기반으로 정보가 통합되는 평균 거리를 구한 attention distance를 나타내보자



초반 레이어에서 부터 attention distance는 큰 값도 존재하고 작은 값도 존재함을 볼 수 있다.

여기서 attention distance는 CNN의 receptive field에 해당한다고 보면된다.

다만 이런 국소적인 어텐션은 hybrid 모델에서는 약하게 나타나는 점을 확인할 수 있다.



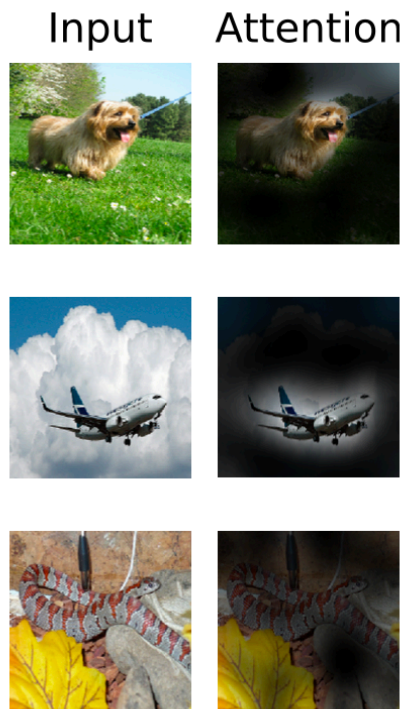
이 말은 결국 모델 앞에 있는 ResNet이 초기 합성곱 커널이 로컬한 패턴을 어느정도 잡아준다는 뜻이다.

다만 합성곱 기반으로는 초반부터 전역적인 부분을 잡아낼 수 없지만

ViT기반은 이러한 전역적인 내용을 잡아 준다는 특징이 있다.

하지만 결국 모두 점점 레이어가 깊어질 수록 전역적인 특징을 잡는다.

Attention Rollout 기법을 이용하면 전체적으로 중요한 부분에 집중하고 있는 것을 확인할 수 있다.



이는 마지막 class 토큰이 어느 부분에 주목하는 지를 시각화한것이다.

Conclusion

기존의 컴퓨터 비전에서 self-attention을 사용하는 연구들과 달리, 초기의 패치 추출 단계를 제외하고는 이미지 특화된 inductive bias를 아키텍처에 도입하지 않았다. 대신, 이미지를 패치들의 시퀀스로 해석하고, 이를 NLP에서 사용되는 표준 트랜스포머 인코더로 처리하였다.

ViT는 대규모 데이터셋에서의 사전학습과 결합될 때 놀라울 정도로 잘 작동한다. 따라서 많은 이미지 분류 데이터셋에서 SOTA 성능과 같거나 이를 능가하며, 사전학습 비용도 상대적으로 저렴하다.

하지만 여전히 많은 과제가 남아 있다. 하나는 ViT를 detection과 segmentation 같은 다른 컴퓨터 비전 과제에 적용해야 한다. 고해상도 이미지가 필요한 태스크에서 아직 ViT는 연산량에 민감한 구조이다.

또 다른 과제는 자기지도(self-supervised) 사전학습 방법을 계속 탐구하는 것이다. 우리의 초기 실험들은 자기지도 사전학습이 성능 향상을 가져옴을 보여주었지만, 자기지도와 대규모 지도학습 사전학습 간에는 여전히 큰 격차가 존재한다.

